VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

Ho Chi Minh City University of Technology

Faculty of Computer Science and Engineering



Deep Learning and Its Applications (CO3133)

Assignment 2 Report

Deep Learning on Large-Scale Data and Specialized Tasks

Semantic Segmentation of Urban Street Scenes:
A Baseline and a Pretrained Model Compared

Instructor: Le Thanh Sach

Semester: 261
Group: 18 – DeepDive
Students: Le Pham Tien Long – 2352688
Ngo Tieu Nghi – 2352799
Ho Minh Nhat – 2352858

HO CHI MINH CITY, 2026

Contents

List of Figures **2**

List of Tables **2**

1 Problem and Data Description **3**

1.1 Problem formulation . . . 3
1.2 Dataset description . . . 3
1.3 Exploratory data analysis . . . 4
1.4 Data splitting . . . 6
1.5 Preprocessing . . . 6

2 Methodology **7**

2.1 Pipeline overview . . . 7
2.2 Baseline and main methods . . . 7
2.3 Training setup . . . 9
2.4 Experimental design . . . 10

3 Implementation Results **11**

3.1 Training behaviour . . . 11
3.2 Quantitative results . . . 12
3.3 Qualitative results . . . 13
3.4 Comparison and discussion . . . 14
3.5 Error analysis . . . 16
3.6 Ablation study . . . 17
3.7 Limitations and conclusion . . . 18

4 AI Usage Disclosure **19**

Code Availability **21**

References **22**

List of Figures

1 Training frames and their label maps; dark regions carry the ignore label. 4
2 Pixel share and frequency per class. 5
3 Frames per city, coloured by partition. 6
4 The pipeline. 7
5 Learning curves; solid validation, dashed training, band one standard deviation. 11
6 Confusion matrix of DeepLabV3-MobileNetV3 on test, rows normalised. 13
7 Correct, difficult and failure cases at image mIoU 0.756, 0.524 and 0.204. 14
8 Per-class IoU on test, hardest class first. 15

List of Tables

1 Class distribution over the training partition. 4
2 Split sizes. 6
3 Shared training protocol. 9
4 The experiment matrix. 10
5 Training behaviour. 11
6 Main comparison. 12
7 Secondary metrics on the test split. 12
8 Every pairwise comparison, on validation mIoU. 14
9 Error taxonomy, as a share of each model's own errors. 16
10 The ablation, on validation IoU. 17
11 Traceability of every reported run. 21

1 Problem and Data Description

1.1 Problem formulation

A vehicle that plans its own path must know not merely that a road is ahead but exactly where the road ends, where the pavement begins and which pixels belong to a pedestrian. Semantic segmentation supplies that map, and it is the perception layer beneath driver assistance, road survey and scene understanding. Cityscapes [1] is the reference benchmark for the urban case, finely annotated at a resolution high enough that thin structures survive.

The task is dense per-pixel classification and the prediction unit is one pixel, so a single 512×1024 frame carries 524,288 predictions. The input is an RGB image $x \in \mathbb{R}^{3\times H\times W}$ and the output a label map $\hat{y} \in \{0,\dots,C-1\}^{H\times W}$ over $C = 19$ evaluated classes. Every model implements a function f_θ returning per-pixel logits $z = f_\theta(x) \in \mathbb{R}^{C\times H\times W}$, and the prediction at pixel (i, j) is $\hat{y}_{ij} = \arg\max_c z_{cij}$. Training minimises the cross-entropy of Equation 1, averaged over the pixels that carry an evaluated label:

$$\mathcal{L}(\theta) = -\frac{1}{|\Omega|}\sum_{(i,j)\in\Omega} \log \frac{\exp(z_{y_{ij}ij})}{\sum_{c=1}^{C}\exp(z_{cij})}, \qquad \Omega = \{(i,j) : y_{ij} \neq 255\}. \tag{1}$$

The restriction to Ω is material rather than formal. Section 1.3 measures 11.8% of all pixels as unlabelled, and a loss that scored them would train the network to reproduce the bonnet of the recording vehicle.

Three difficulties dominate, and all three are properties of the data rather than of the objective. The classes span almost three orders of magnitude in pixel share, so an objective that weights pixels equally is governed by whatever covers the most ground. Scarcity arrives in two forms that fail for different reasons, quantified in Section 1.3, and no single remedy addresses both. And the output must retain spatial resolution: a classifier may collapse the image to a vector, whereas here the network has to carry location all the way to the prediction, which is what makes the decoder, or the dilated backbone that replaces it, the architectural question.

1.2 Dataset description

Cityscapes [1] is distributed by its authors at `https://www.cityscapes-dataset.com` under a licence permitting non-commercial research and educational use, and requires a registered account. The two packages used here, `leftImg8bit_trainvaltest` and `gtFine_trainvaltest`, were fetched on 7 October 2026. The frames were recorded from a moving vehicle across 27 cities in Germany, France and Switzerland, over several months, in daylight and fair weather. The fine annotations were drawn as per-pixel polygons by human annotators and quality-controlled by the dataset authors, so the labels are expert rather than crowd-sourced.

The fine-annotation set holds 5,000 frames at 1024×2048: 2,975 in the official train split across 18 cities, 500 in val across 3, and 1,525 in test across 6 whose labels are withheld for the public leaderboard. Images are 8-bit PNG, label maps single-channel PNG of label ids. Storage is 11 GB of images and 808 MB of labels, from 11.3 GB of archives that can be deleted once unpacked.

The annotation defines 34 label ids of which 19 are evaluated; the remainder, including void regions, the ego vehicle and the rectification border, map to the ignore label 255. The 19 evaluated classes are road, sidewalk, building, wall, fence, pole, traffic light, traffic sign, vegetation, terrain, sky, person, rider, car, truck, bus, train, motorcycle and bicycle, and their distribution is the subject of Section 1.3.

Three limitations bound what these results can claim. The recording conditions are narrow: daylight, fair weather, European cities and one camera, so neither the ranking nor the error structure should be assumed to hold at night, in rain or on another continent. The class taxonomy embeds decisions that are conventions rather than visual facts, the boundary between terrain and vegetation among them, so part of the residual error is irreducible. And the ignore label is not uniformly distributed: 7.7% of every frame is unusable regardless of content, because the bonnet occupies the bottom and the rectification border the edges, which removes image area exactly where the road surface nearest the vehicle would be.

1.3 Exploratory data analysis

Every measurement in this section is taken over all 2,406 label maps of the training partition rather than a sample. Figure 1 shows three frames with their annotations.



Figure 1: *Training frames and their label maps; dark regions carry the ignore label.*

Mask quality. The annotations are complete in the sense that matters: every frame has a label map of identical resolution, every pixel carries a valid label id, and no frame was discarded. What varies is how much of each frame is evaluable. 11.8% of all pixels carry the ignore label, of which 7.7% is the structural loss described above and the remainder genuine void annotation. No interpolation is ever applied to a label map, for the reason given in Section 1.5.

Pixel distribution. Table 1 and Figure 2 report the distribution three ways, because the mean pixel share alone is misleading. Road, building and vegetation together occupy 75.8% of evaluated pixels, so a model that predicted nothing but those three would already reach roughly 0.76 pixel accuracy. This is the measured reason mIoU is the primary metric and pixel accuracy decides nothing: mIoU averages over classes, so the sixteen remaining classes cannot be hidden by the three largest.

Table 1: *Class distribution over the training partition.*

Class	Pixels	Images	Area	Crops	Class	Pixels	Images	Area	Crops
road	36.80%	98%	33.56%	93%	wall	0.74%	36%	0.69%	17%
building	22.92%	99%	20.16%	82%	traffic sign	0.56%	94%	0.31%	47%
vegetation	16.07%	97%	13.15%	71%	bicycle	0.39%	54%	0.23%	26%
car	6.78%	95%	4.18%	71%	truck	0.31%	12%	0.45%	4%
sidewalk	6.16%	94%	4.67%	79%	bus	0.22%	9%	0.66%	4%
sky	3.88%	90%	2.97%	25%	train	0.21%	4%	0.91%	2%
person	1.32%	79%	0.52%	46%	traffic light	0.20%	54%	0.18%	23%
pole	1.17%	99%	0.83%	77%	rider	0.13%	33%	0.13%	13%
terrain	1.12%	55%	0.60%	30%	motorcycle	0.11%	18%	0.13%	6%
fence	0.92%	43%	0.81%	19%					

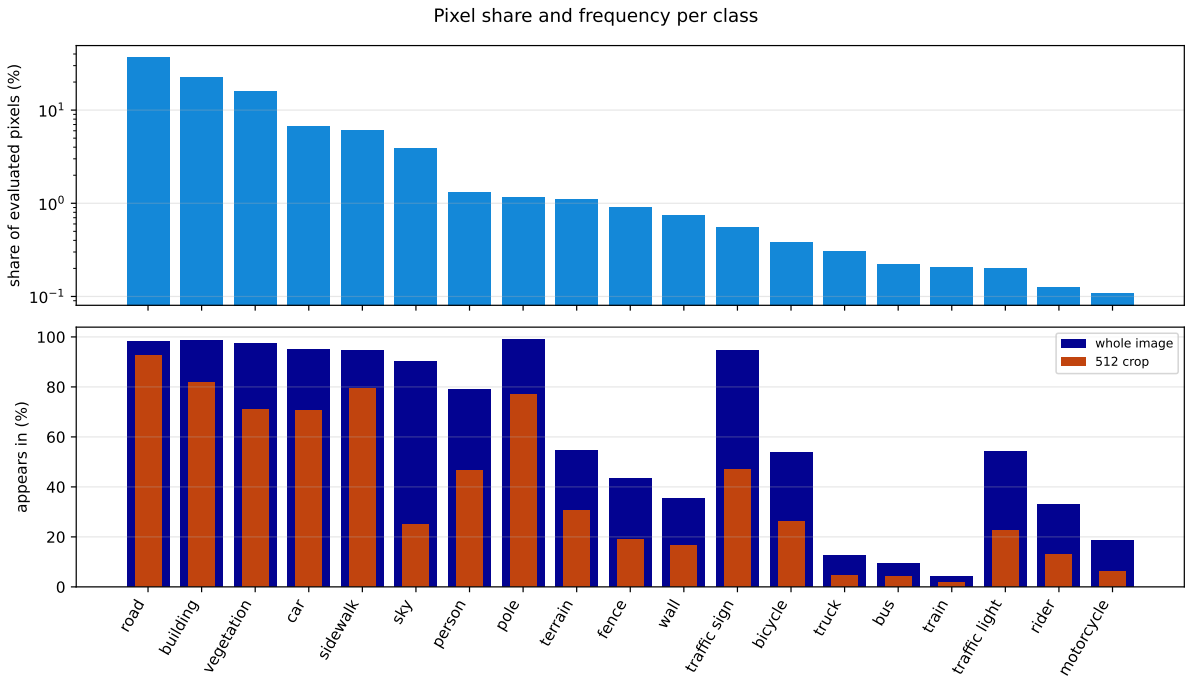


Figure 2: *Pixel share and frequency per class.*

Region size, and the two forms of scarcity. Comparing the *pixels* and *images* columns of Table 1 separates two failure modes that a single frequency count conflates.

The first is thin structures. Poles account for 1.17% of evaluated pixels but appear in 99% of images, and traffic signs for 0.56% of pixels in 94% of images. Examples are therefore abundant; what is scarce is pixel coverage, so the errors these classes produce are boundary errors rather than the result of too few training instances.

The second is large objects that are seldom photographed. Trains occupy 0.21% of pixels in 4% of images and buses 0.22% in 9%. When one is present, however, its median area is 0.91% and 0.66% of the frame, four and three times its average share. A reweighted loss addresses the second form and not the first, which is what makes the loss ablation of Section 3.6 worth running.

What cropping does to the prior. The last column of Table 1 samples one random 512×512 crop per image, which is what a training batch actually sees. A crop covers an eighth of the resized frame, and it sharpens the second form of scarcity: trains fall from 4% of images to 2% of crops and buses from 9% to 4%. Sky is the clearest case and is not a rare class at all, falling from 90% of images to 25% of crops, because it occupies a band at the top of the frame rather than a compact region. Cropping therefore reshapes the class prior spatially and not only by area, and the effective training distribution is the crop column rather than the pixel column.

Figure 3 completes the picture across the split unit, showing that no city is large enough to dominate the partition it belongs to.

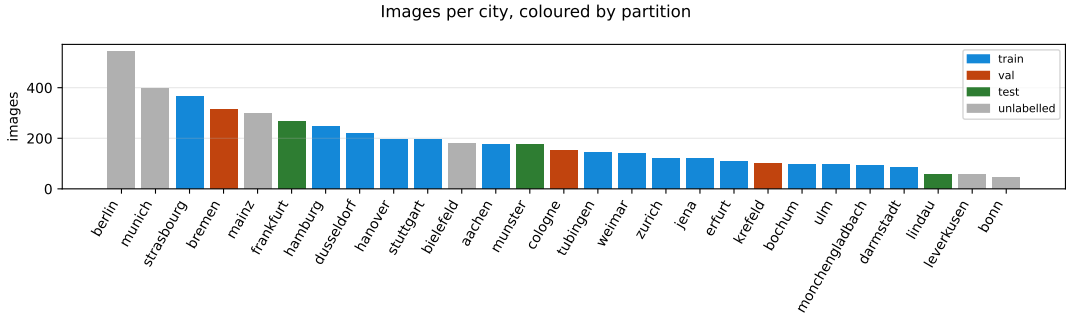


Figure 3: *Frames per city, coloured by partition.*

1.4 Data splitting

The split unit is the city, not the frame. Consecutive frames from one street share buildings, parked cars and road surface, so splitting by frame would place near-duplicates on both sides of the boundary and let a model score by recognising a scene it had trained on. The official Cityscapes partitions already share no city, and the split used here preserves that property.

The official test labels are withheld, so the official validation partition serves as the test set. A checkpoint cannot then be selected on it without selecting on the test set, so the three cities that select the checkpoint are held out of the official train partition instead. Table 2 gives the result.

Table 2: *Split sizes.*

Partition	Source	Cities	Images
Train	official train	15	2,406
Validation	official train, held out at seed 0	3	569
Test	official val	3	500

Seed 0 holds out Bremen, Cologne and Krefeld for validation; the test cities are Frankfurt, Lindau and Münster, as the official partition fixes them. No stratification is applied, because the unit is a city and every city contains every common class; Figure 3 shows the resulting sizes are comparable. There are no subset selection rules: the full fine-annotation set is used.

The split is generated once, committed as a file, and read by every run, so it is identical across models, seeds and machines. Three rules prevent leakage, each closing a different route. The split file is read rather than regenerated, so no run can silently resample it. Normalization statistics never come from the data a model is scored on, as Section 1.5 describes. And the test partition is read once per configuration, after model selection concludes, so it never participates in a decision; Section 3.2 reports both splits and the gap between them.

1.5 Preprocessing

No cleaning is required: every frame has a complete label map, and the dataset's own documentation records no invalid annotations. Invalid and unevaluated content is handled by the label mapping rather than by discarding data. The 34 label ids are remapped to the 19 train ids with everything else sent to the ignore label, and that table was read from the dataset's own toolkit rather than retyped, since a single misplaced row would corrupt every metric in this report without raising an error.

Images are resized from 1024×2048 to 512×1024 by bilinear interpolation. **Label maps are resized by nearest neighbour**, never bilinearly: averaging label ids 7 and 9 would produce 8, a class present in neither neighbour, so interpolation would invent annotations. The training loader then takes a random 512×512 crop and applies a horizontal flip with probability 0.5; the same geometric operation is applied to the image and its mask. Validation and test loaders are deterministic and consume the whole resized

frame, so evaluation is never measured on crops. No padding is used, since every frame has identical dimensions.

Images are scaled to $[0, 1]$ and standardised per channel with the ImageNet statistics $\mu = (0.485, 0.456, 0.406)$ and $\sigma = (0.229, 0.224, 0.225)$. These are the statistics the pretrained backbones were fitted under, and feeding a differently normalised batch to a pretrained network degrades it silently with no error to notice. The training partition itself measures $\mu = (0.288, 0.325, 0.285)$ and $\sigma = (0.186, 0.188, 0.185)$, which is recorded in the proposal but deliberately not used: adopting it would improve the baseline's input conditioning while corrupting the pretrained model's, and the comparison requires that both models receive identical input. Both models therefore see the same tensors, and neither normalization is ever computed from validation or test data.

A batch leaving the training loader is a pair of tensors: images of shape $(8, 3, 512, 512)$ in `float32` spanning $[-2.118, 2.640]$, and label maps of shape $(8, 512, 512)$ in `int64` holding values in $\{0, \dots, 18\} \cup \{255\}$. The observed range confirms the standardisation, since $(0 - 0.485)/0.229 = -2.118$ is the value of a black pixel in the red channel and $(1 - 0.406)/0.225 = 2.640$ that of a white one in the blue. In the batch sampled here 10.44% of label pixels carry the ignore label, consistent with the 11.8% measured over the whole partition. A validation batch is identical in dtype and range but of shape $(8, 3, 512, 1024)$, since it is not cropped.

2 Methodology

2.1 Pipeline overview

Figure 4 shows the path from the raw archives to a reported number. Every configuration in this report takes the identical path; only the blocks named in Section 2.4 change, one at a time, which is what makes a difference in score attributable to the factor varied rather than to the pipeline around it.

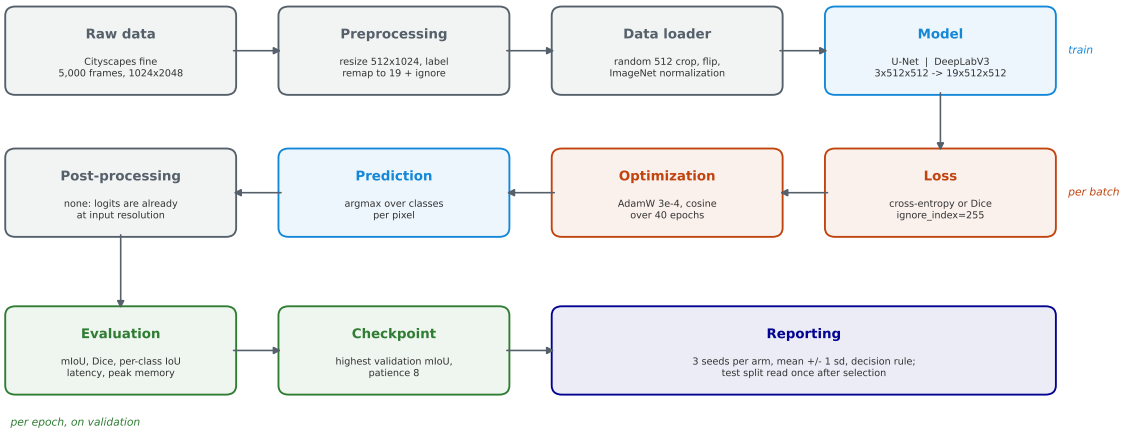


Figure 4: *The pipeline.*

Two stages deserve a note. Post-processing is empty by design: both models emit logits at the input resolution, so the prediction is an `argmax` and nothing is upsampled, smoothed or thresholded afterwards. And evaluation runs twice on different terms. Validation is computed every epoch and selects the checkpoint; the test split is read once per configuration, after selection has concluded.

2.2 Baseline and main methods

Both models consume the identical input representation, a $3 \times 512 \times 512$ crop normalised as Section 1.5 describes, and emit $19 \times 512 \times 512$ logits scored by the same metrics. They differ in where their features

come from: one learns everything from 2,406 frames, the other starts from weights fitted on a far larger corpus.

U-Net, the baseline. A symmetric encoder and decoder [2] written from scratch, base width 32 and depth 4, with no pretrained weights. Each encoder stage applies two padded 3×3 convolutions, each batch-normalised and rectified, then halves the resolution by max pooling; widths double at every level, $32 \to 64 \to 128 \to 256$, and a 512-wide block bridges the bottom. The decoder mirrors this, upsampling by transposed convolution and ending in a 1×1 convolution to the 19 logits. Formally, writing e_ℓ for the encoder map at level ℓ and d_ℓ for its decoder counterpart,

$$e_\ell = \mathrm{DoubleConv}(\mathrm{MaxPool}_2(e_{\ell-1})), \qquad d_\ell = \mathrm{DoubleConv}([\,\mathrm{ConvT}_2(d_{\ell+1});\, e_\ell\,]), \tag{2}$$

where $[\cdot\,;\cdot]$ concatenates along channels. The skip connection in Equation 2 is the point of the architecture rather than a detail of it: four poolings reduce a 512-pixel crop to 32 pixels, which carries enough context to say what a region is but is far too coarse to say where its boundary runs, so each decoder stage places the upsampled semantics back onto the encoder detail that produced them. Padding keeps every convolution at its input size, so a skip always meets a map of exactly matching resolution. Convolutions carry no bias because each is followed by batch normalisation, which recentres the activations and makes the bias redundant. The model holds 7,763,635 parameters, all trainable. Its strength is that it is fully determined by this data; its limit is that it must learn every edge and texture detector from 2,406 frames.

DeepLabV3, the pretrained model. Torchvision's DeepLabV3 [3] with a MobileNetV3-Large backbone [4], pretrained on a COCO subset [5] carrying the twenty VOC classes, over an ImageNet-pretrained [6] classification backbone. It replaces the U-Net's decoder with atrous convolution: the backbone's final stages use dilated kernels so the feature map is never reduced below a stride of 8, and an atrous spatial pyramid pooling module gathers context at several dilation rates in parallel before a 1×1 convolution produces the logits, which a single bilinear interpolation returns to the input resolution. The published head predicts 21 classes, so its final 1×1 convolution is replaced by a 19-class one; the backbone and the pooling module transfer unchanged, since they carry the learned features. The model holds 11,024,963 parameters.

Two implementation details matter for the comparison. The auxiliary head the architecture can carry is removed rather than computed and discarded, so training pays nothing for an output no loss reads. And the normalization is taken from the weights' own transform rather than assumed, for the reason given in Section 1.5.

Fine-tuning procedure. A single stage, with no schedule of its own. The final convolution is replaced for the 19 classes, and every parameter trains from the first epoch at one learning rate: no layer is unfrozen part way through and no layer carries a separate rate. This is deliberate, because it is the procedure the U-Net also receives, so Section 3.4 compares models rather than training schedules. The controlled experiment varies exactly this choice by freezing the backbone, which leaves 8,053,011 of the 11,024,963 parameters trainable. That the frozen arm still trains 73% of the model is not a mistake but a property of the architecture: the pooling module and classifier hold 8.05M parameters against the backbone's 2.97M, so freezing a MobileNetV3 backbone removes far less than the word suggests. Section 3.4 returns to this.

Backbone variant. The extension replaces MobileNetV3-Large with ResNet50 [7], giving 39,638,355 parameters, 3.6 times the baseline backbone, under an otherwise identical procedure.

Losses and metrics. The default objective is the pixel cross-entropy of Equation 1. The ablation replaces it with a soft Dice loss [8] computed over the same labelled pixels Ω,

$$\mathcal{L}_{\mathrm{Dice}}(\theta) = 1 - \frac{1}{C}\sum_{c=1}^{C} \frac{2\sum_{(i,j)\in\Omega} p_{cij}\,\mathbf{1}[y_{ij}=c] + \varepsilon}{\sum_{(i,j)\in\Omega} p_{cij} + \sum_{(i,j)\in\Omega}\mathbf{1}[y_{ij}=c] + \varepsilon}, \qquad \varepsilon = 1, \tag{3}$$

with $p = \mathrm{softmax}(z)$. The two differ in what they weight. Cross-entropy averages over pixels, so a class contributes in proportion to the ground it covers; Dice averages over classes, so a pole counts as much as the road. Equation 3 averages over all C classes whether or not the batch contains them, a property Section 3.6 shows to be consequential.

Mean intersection over union is the primary metric, with Dice, per-class IoU and pixel accuracy secondary. Both are computed from one running 19×19 confusion matrix accumulated over a split, never from concatenated masks, since a validation pass at 512×1024 holds 298 million pixels. For class c with true positives TP_c, false positives FP_c and false negatives FN_c,

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \mathrm{Dice}_c = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{4}$$

and mIoU is the mean over the classes present in the split: a class absent from the ground truth is excluded from the average rather than scored zero. Pixel accuracy is recorded but never decides a comparison, for the reason measured in Section 1.3.

Implementation. The U-Net, the Dice loss, the confusion matrix and every metric derived from it, the training loop, the evaluation, the experiment runner and the figures are written by the group. DeepLabV3 and its pretrained weights come from torchvision; layer primitives (`nn.Conv2d`, `nn.ConvTranspose2d`, `nn.BatchNorm2d`, `nn.MaxPool2d`), cross-entropy, AdamW and the cosine schedule come from PyTorch [9]; the label-id table is read from `cityscapesscripts`; the plots come from matplotlib.

2.3 Training setup

Every configuration is trained under the identical protocol of Table 3. A run that deviates from it is repeated rather than reported.

Table 3: *Shared training protocol.*

Setting	Value
Input	Resized to 512×1024; training takes a random 512×512 crop, evaluation the whole resized frame
Optimizer	AdamW [10], $\eta = 3 \times 10^{-4}$, weight decay $\lambda = 0.05$, over the parameters that require gradients
Batch size	8, giving 301 steps per epoch
Epochs	40 maximum
Scheduler	Cosine annealing, stepped once per epoch, period equal to the 40 configured epochs
Regularization	Weight decay; random crop and horizontal flip on the training loader only; no dropout
Early stopping	After 8 epochs without an improvement in validation mIoU
Checkpoint criterion	Highest validation mIoU
Seeds	0, 1 and 2 for every configuration entering a comparison
Hardware	Apple M4 Pro, 14 cores, 48 GB unified memory, MPS backend
Software	Python 3.12.0, PyTorch 2.14.0, torchvision 0.29.0
Mixed precision	Not used; all runs are full `float32`
Training time	25.5 hours over 13 runs, from 0.50 h to 8.22 h per run

Hyperparameter selection. None was performed. The optimizer, schedule and budget were fixed in advance from values conventional for AdamW on a dense prediction task and applied unchanged to every arm, since tuning them per arm would confound the comparison with unequal search effort. Each configuration ran once, at a setting fixed before training, and an experiment changes exactly one of those settings. The search effort is therefore zero and identical across arms, and every score in this report is a score under a common protocol at an untuned configuration rather than the best that configuration could reach.

Early stopping interacts with the schedule in one way worth stating: it does not shorten the cosine period, so a run that stops early ends above the minimum learning rate rather than at it.

Reproducibility, and its limit on this hardware. Each seed sets the Python, NumPy and PyTorch generators together, and every run records the configuration it resolved to, the software versions it ran under and the commit it was produced at. One limit is imposed by the hardware and is reported

rather than hidden. Deterministic algorithms are requested, but the Apple GPU backend provides no deterministic kernel for the scatter-add that DeepLabV3 reaches in the backward pass of the bilinear interpolation returning its logits to full resolution, so PyTorch warns and proceeds nondeterministically rather than failing. Measured at a fixed seed, the U-Net reproduces bit-exactly while DeepLabV3 diverges by about 4×10^{-7} per step. The seed-to-seed spread reported throughout Section 3 consequently includes this nondeterminism for the DeepLabV3 arms, and rerunning one of those seeds will not reproduce its number exactly.

2.4 Experimental design

Four experiments are reported. Each was stated in full, with its hypothesis and its decision criterion, before the runs were launched. Table 4 summarises them: each changes one thing and holds the rest of Table 3 fixed.

Table 4: *The experiment matrix.*

Role	Factor varied	Seeds
Model comparison	U-Net trained from scratch against the pretrained DeepLabV3	$3 + 3$
Controlled experiment	Backbone frozen against fully fine-tuned	3
Ablation	Cross-entropy against Dice loss	3
Backbone extension	MobileNetV3-Large against ResNet50	1

The model comparison. *Hypothesis:* the pretrained DeepLabV3 outperforms the U-Net trained from scratch at comparable capacity, 11.0M against 7.8M parameters, because 2,406 training images are too few to learn general features from scratch. *Varies:* the model. *Fixed:* the split, the protocol, the loss, the normalization and the seed set. *Metrics:* mIoU primary; Dice, per-class IoU, parameter count, seconds per epoch and inference latency secondary.

The controlled experiment: frozen against fully fine-tuned backbone. *Hypothesis:* fine-tuning every parameter outperforms training the head on a frozen backbone, because features fitted to object-centric photographs are not adapted to street scenes; the gap is widest on thin classes such as pole and traffic sign, whose boundaries depend on low-level features the frozen arm cannot change. *Varies:* whether the backbone receives gradients. *Fixed:* backbone, pretrained weights, head, learning rate and the rest of the protocol. *Metrics:* mIoU and per-class IoU, trainable parameters, seconds per epoch. One consequence of freezing is stated in advance: the backbone stops learning but stays in training mode, so its batch-normalization running statistics still adapt to Cityscapes. The frozen arm fixes every learned weight but not those statistics.

The ablation: cross-entropy against Dice loss. *Hypothesis:* Dice raises IoU on classes that are rare but large when present, namely train, bus and truck, because it weights every class equally regardless of pixel count; it does not help thin classes, whose errors lie at boundaries rather than in how often the class is seen. *Varies:* the training objective. *Fixed:* the fully fine-tuned DeepLabV3-MobileNetV3 and the protocol. *Metrics:* mIoU and Dice, and per-class IoU for train, bus, truck, pole and traffic sign. The hypothesis holds only if the per-class rule separates the two arms on at least two of train, bus and truck.

The backbone extension. *Hypothesis:* ResNet50 outperforms MobileNetV3-Large at 3.6 times the parameters and roughly 7 times the cost per step. *Varies:* the backbone. *Fixed:* full fine-tuning, cross-entropy and the protocol. *Metrics:* mIoU, parameter count, seconds per epoch, inference latency. Run at seed 0 only, so it is reported as indicative and cannot substitute for the controlled experiment or the ablation.

Conclusion criterion. Fixed before any result was seen and applied to every comparison in this report. Configuration A is reported as better than B only when the mean validation mIoU of A exceeds that of B *and* the two intervals of mean $\pm$ one standard deviation across the three seeds do not overlap. Overlapping intervals mean the two are indistinguishable under this protocol, which is itself a result and is not resolved by picking the higher mean. A claim about a single class applies the same rule to that class's IoU. A configuration run at one seed supports no claim of superiority.

3 Implementation Results

3.1 Training behaviour

Figure 5 shows the learning curves, each averaged over its seeds. The left panel is labelled *training objective* rather than loss because the ablation arm minimises Equation 3 and the others Equation 1; the two are not on a common scale and should be read only against their own arm.

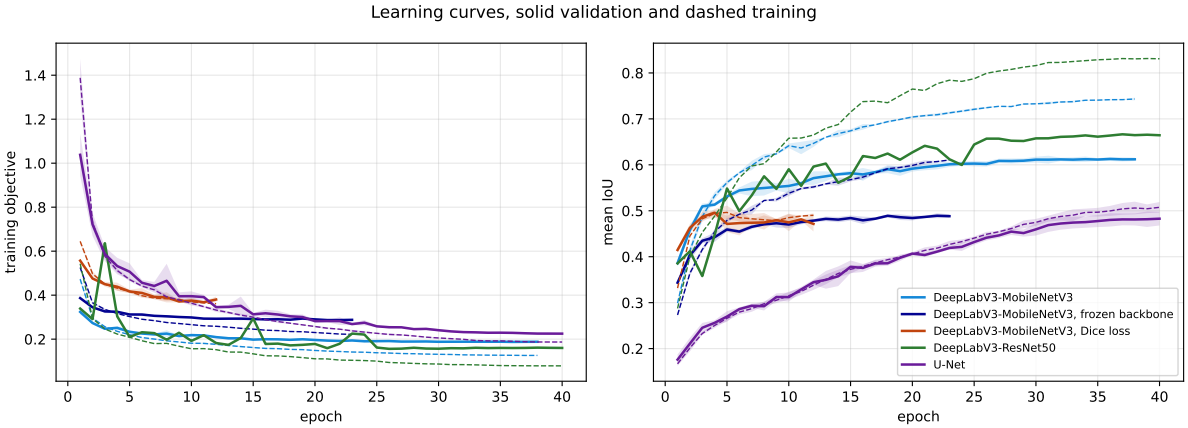


Figure 5: *Learning curves; solid validation, dashed training, band one standard deviation.*

The budget was the right size, for three of the five arms. The pretrained DeepLabV3 runs stopped at epochs 38, 39 and 40 with their best epoch at 30, 31 and 35, and their validation curve is flat from about epoch 32, rising from 0.615 to 0.616 over the last eight epochs. The 40-epoch budget was therefore adequate without being generous. The ResNet50 run behaved the same way, best at epoch 37 of 40.

The U-Net is the exception, and its number is a lower bound. Its best epochs were 38, 40 and 39 of 40, and its validation mIoU rises monotonically to the last epoch, 0.446 at epoch 28, 0.460 at 32, 0.466 at 36 and 0.468 at 40. It was still improving when the budget ran out. Every U-Net figure in this report is therefore a floor rather than a converged score, which Section 3.7 returns to.

Overfitting is present but mild, and is not what separates the arms. Table 5 gives the gap between training and validation mIoU at the selected epoch. The pretrained arms sit between 0.11 and 0.16, and validation mIoU never turns downward for them; the U-Net's gap is only 0.021, which is consistent with a model still underfitting at epoch 40 rather than one generalising unusually well. The Dice arm is the instructive case: its gap is −0.002, so it does not overfit at all, it simply stops improving.

Table 5: *Training behaviour.*

Configuration	Epochs run	Best epoch	Gap	s/epoch	Hours
DeepLabV3-MobileNetV3	38 / 39 / 40	30 / 31 / 35	0.121	140.4	1.52
DeepLabV3-MobileNetV3, frozen backbone	23 / 32 / 40	15 / 24 / 36	0.110	95.3	0.84
DeepLabV3-MobileNetV3, Dice loss	12 / 12 / 12	4 / 4 / 4	−0.002	150.5	0.50
DeepLabV3-ResNet50	40	37	0.165	739.9	8.22
U-Net	40 / 40 / 40	38 / 40 / 39	0.021	260.2	2.89

Early stopping. It fired for three arms and not for the U-Net or ResNet50. The frozen arm stopped at 23, 32 and 40 epochs, a spread of 17 epochs from nothing but the seed, which is a reminder that the stopping epoch is a far noisier quantity than the score it protects. The Dice arm stopped at epoch 12 on all three seeds, having peaked at epoch 4 on all three; Section 3.6 explains why.

Training time. The 13 runs took 25.5 hours in total on the single machine of Table 3, run consecutively so that the timings compare with each other, from 0.50 hours for a Dice run that stopped at epoch 12 to 8.22 hours for ResNet50.

Implementation issues found and fixed. Four, each of which would have corrupted a reported number. The qualitative-example selection kept only the first six images of the split before ranking them, so the "failure case" was the worst of an arbitrary six rather than of the split; after the fix the selected range widened from 0.58 down to 0.42, and after it from 0.77 down to 0.29 image mIoU. The selected examples were then discarded before any overlay was written, so the figures could never have been produced. Inference latency was measured with autograd active, which is not what inference does; the measured effect on this backend was within noise, 14.82 against 15.71 ms, but it would dominate peak memory on CUDA. And two early draft runs used ten epochs: because the cosine period equals the configured epoch count, their learning-rate trajectory differs from a 40-epoch run at every step, so they were deleted rather than pooled.

3.2 Quantitative results

Table 6 reports every mandatory comparison quantity. Validation mIoU is the quantity the decision rule of Section 2.4 is applied to, because it is what selected the checkpoints; the test column is reported once, afterwards, and decides nothing. The evaluation reproduces the training-time validation mIoU exactly for all 13 runs, to four decimal places, which confirms that the checkpoint written during training is the one scored here.

Table 6: *Main comparison.*

Configuration	Params	s/ep	h	ms@1	ms@8	GB	Val mIoU	Test mIoU
DeepLabV3-ResNet50	39,638,355	739.9	8.22	72.3	76.8	4.37	*0.6665*	*0.7076*
DeepLabV3-MobileNetV3	11,024,963	140.4	1.52	15.2	8.2	3.41	$\mathbf{0.6136 \pm 0.0051}$	$\mathbf{0.6180 \pm 0.0049}$
frozen backbone	8,053,011	95.3	0.84	14.7	8.3	3.41	0.4961 ± 0.0049	0.5003 ± 0.0088
Dice loss	11,024,963	150.5	0.50	14.9	8.2	3.41	0.4958 ± 0.0045	0.4983 ± 0.0123
U-Net, from scratch	7,763,635	260.2	2.89	22.6	22.0	6.57	0.4836 ± 0.0154	0.4702 ± 0.0071

Table 7: *Secondary metrics on the test split.*

Configuration	Dice	Pixel accuracy	Error rate
DeepLabV3-ResNet50	*0.8181*	*0.9509*	4.9%
DeepLabV3-MobileNetV3	$\mathbf{0.7441 \pm 0.0048}$	0.9329	6.7%
frozen backbone	0.6336 ± 0.0091	0.8959	10.4%
Dice loss	0.6225 ± 0.0167	0.8982	10.2%
U-Net, from scratch	0.5710 ± 0.0113	0.9202	8.0%

Three observations follow from the two tables before any comparison is drawn.

Pixel accuracy and mIoU disagree, exactly as the EDA predicted. In Table 7 the U-Net reaches 0.9202 pixel accuracy, higher than both the frozen arm (0.8959) and the Dice arm (0.8982), while scoring lower than both on mIoU. A metric dominated by the three classes that cover 75.8% of the ground ranks these three models in the opposite order to the metric that weights all nineteen equally. This is the measured justification for the choice made in Section 1.3, and it is why pixel accuracy appears in this report only as a secondary column.

Validation does not overstate test here; for four arms of five it understates it. Test mIoU exceeds validation mIoU by 0.0043 for DeepLabV3, 0.0042 for the frozen arm, 0.0024 for Dice and 0.0412 for ResNet50. Only the U-Net is lower on test, by 0.0134. This runs against the usual expectation and has a simple cause: with a city-level split, the three test cities are simply a different sample of difficulty from the three validation cities, and that partition-to-partition difference is of the same order as, or larger than, the seed-to-seed spread. Pixel accuracy moves the other way for every arm, falling on test while mIoU rises, which is consistent with the test cities holding a class mix that is harder per pixel but better populated in the rarer classes. What transfers between the two columns is therefore the margin between two arms rather than the absolute level.

The seed spread is small except for the U-Net. The standard deviation across seeds is 0.0049 to 0.0088 validation mIoU for the three pretrained arms but 0.0154 for the U-Net, three times larger, with individual seeds at 0.4687, 0.4824 and 0.4995. Training from scratch is markedly less reproducible than fine-tuning here. The U-Net's best seed alone would have been indistinguishable from the frozen arm;

its worst would have lost under the rule. A single-seed comparison could therefore have reported either outcome, which is the concrete argument for the three-seed protocol.

3.3 Qualitative results

Figure 6 gives the confusion matrix of the pretrained DeepLabV3 on the test split, rows normalised because the class supports span three orders of magnitude.

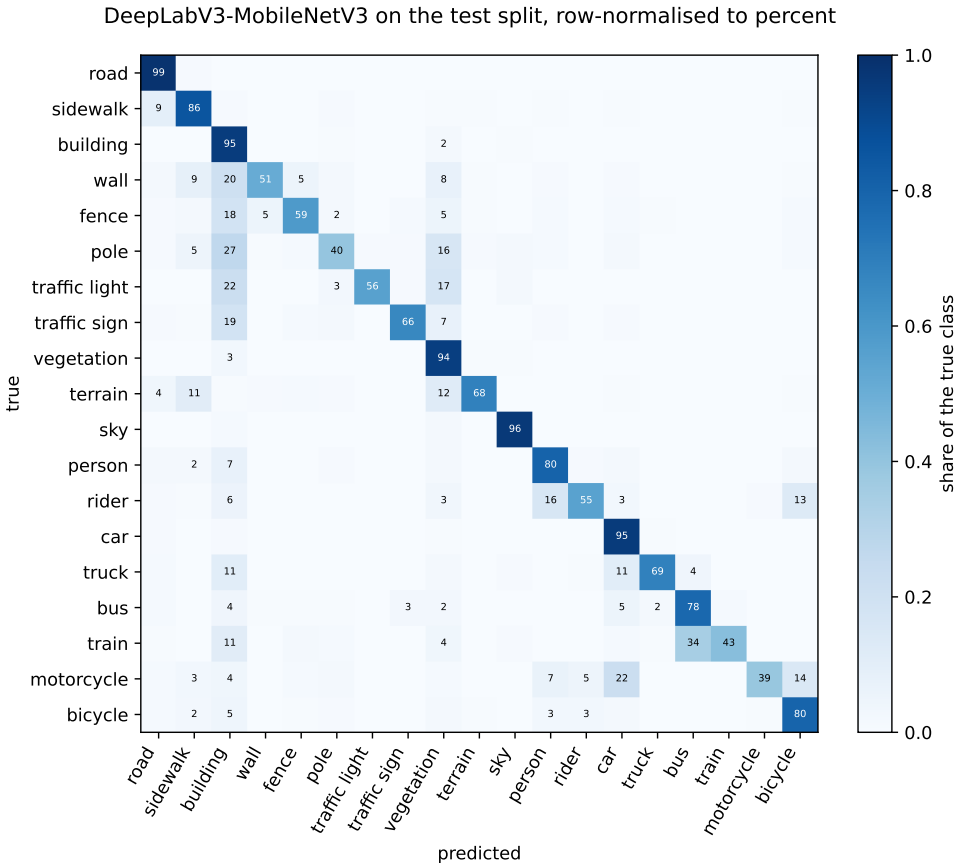


Figure 6: *Confusion matrix of DeepLabV3-MobileNetV3 on test, rows normalised.*

Six examples per run are selected without human choice: the split is ranked by per-image mIoU and sampled at even ranks, so the best image is the correct case, the worst the failure, and the ranks between them the difficult ones. For DeepLabV3 at seed 0 on test this spans 0.7556 down to 0.2041 image mIoU. Figure 7 shows three of the six.

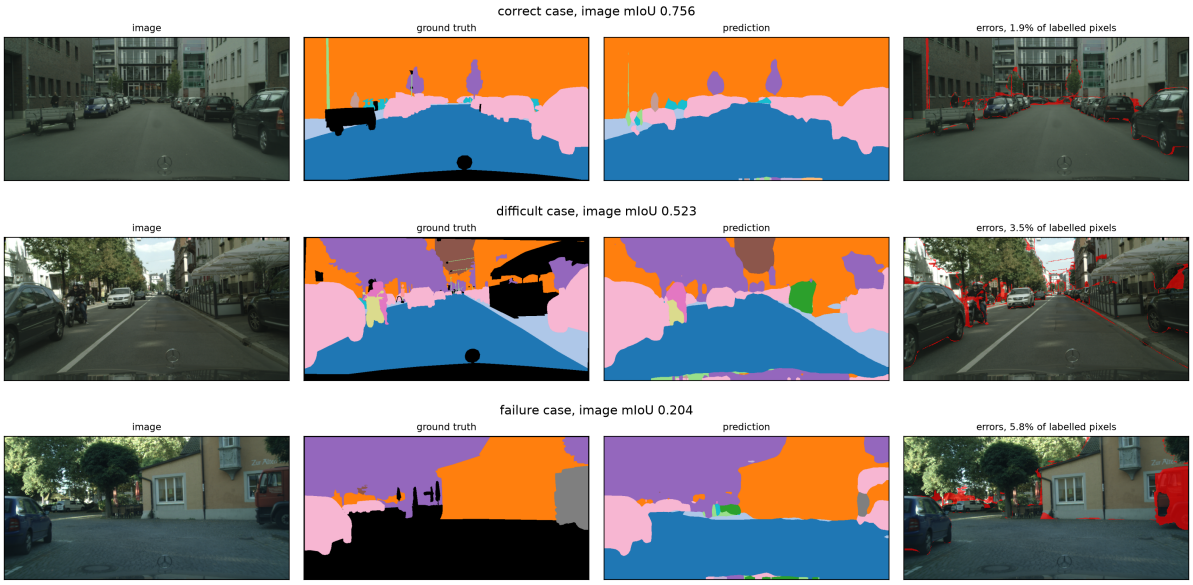


Figure 7: *Correct, difficult and failure cases at image mIoU* 0.756*,* 0.524 *and* 0.204*.*

The three rows say different things. In the correct case the large regions are recovered almost exactly and the visible error is a thin outline tracing every boundary: the model knows what each region is and is imprecise about where it ends. The difficult case adds the failure mode the confusion matrix quantifies, with poles and signs partially absorbed into the building behind them.

The failure case is the most informative, and not for the reason its mIoU suggests. Its error panel reports that only 5.8% of labelled pixels are wrong, a *lower* error rate than the split average of 6.7%, yet it is the worst image in the split at 0.204 mIoU. The cause is visible in the ground truth: most of the courtyard carries the ignore label, so little of the frame is scored, and the few classes that are present have small support. A per-image mean over classes is extremely sensitive when each class is represented by a handful of pixels. The lesson is that per-image mIoU and per-image pixel error are only loosely coupled, and that "worst image by mIoU" selects for images with little labelled content as much as for images the model misunderstands.

3.4 Comparison and discussion

Applying the criterion of Section 2.4 to the validation column of Table 6 gives Table 8.

Table 8: *Every pairwise comparison, on validation mIoU.*

Comparison	Verdict	Margin
DeepLabV3 vs. U-Net	DeepLabV3 better	0.1301
DeepLabV3 vs. frozen backbone	Full fine-tuning better	0.1175
DeepLabV3 vs. Dice loss	Cross-entropy better	0.1178
U-Net vs. frozen backbone	**Indistinguishable**	0.0126
U-Net vs. Dice loss	**Indistinguishable**	0.0123
Frozen backbone vs. Dice loss	**Indistinguishable**	0.0003
Anything vs. ResNet50	Indicative only, one seed	not applicable

The headline is a single large effect, and three ways of losing it. One configuration, the fully fine-tuned pretrained model, separates from everything else. The other three three-seed arms are mutually indistinguishable, clustered within 0.0126 validation mIoU of one another despite differing in architecture, in pretraining, and in objective. Freezing the backbone, swapping cross-entropy for Dice, and training a different architecture from scratch are three unrelated interventions, and all three land in the same place. The frozen arm and the Dice arm differ by 0.0003. This is the strongest result in the report: on this dataset at this budget, what matters is not which of several reasonable choices is made but whether the model starts from adapted pretrained features at all.

Where the difference appears. Figure 8 breaks the aggregate down. Against the frozen arm, full fine-tuning is better on *all nineteen* classes with non-overlapping intervals, so nothing about the frozen arm is preserved. But the gap is very unevenly distributed: 0.027 on road, 0.044 on car and 0.048 on building, against 0.215 on truck, 0.213 on traffic sign, 0.173 on traffic light and 0.166 on bus. A frozen object-centric backbone still supports the classes that fill a street scene and loses precisely the classes that do not.

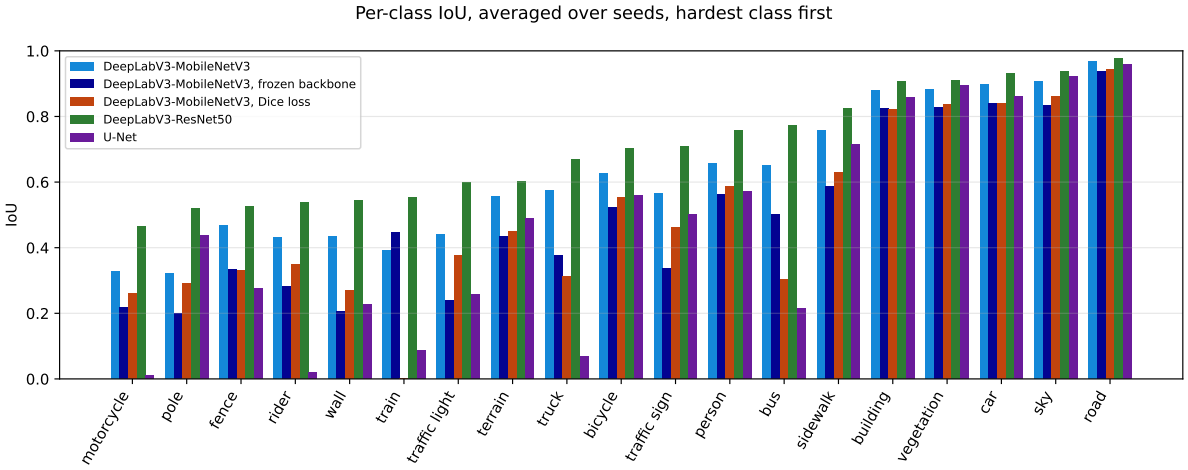


Figure 8: *Per-class IoU on test, hardest class first.*

This partly supports the hypothesis of the controlled experiment and partly refutes it. The prediction that the gap would be widest on thin classes is right for traffic sign and traffic light, which are second and third largest, but the single widest gap is truck, a rare-but-large class, and pole, the thin class named explicitly in the hypothesis, sits mid-table at 0.128. The correct statement is narrower than the one pre-registered: freezing costs most on the classes that are *rare in the training distribution*, whether they are rare because they are thin in area or because they are seldom photographed, and costs least on the three classes that cover most of the ground.

Pretraining is not uniformly better, which the aggregate hides. The U-Net beats the pretrained DeepLabV3, with non-overlapping intervals, on three classes: pole (0.4126 against 0.3277), sky (0.9332 against 0.9211) and vegetation (0.8823 against 0.8740). Pole is the interesting one, a 26% relative margin for the weaker model. The cause is architectural rather than about features. DeepLabV3 never resolves below a stride of 8 and returns its logits to full resolution by one bilinear interpolation, which cannot reconstruct a structure a few pixels wide; the U-Net's skip connections carry full-resolution detail into the decoder, which is exactly what a pole needs.

The from-scratch baseline is therefore stronger at localisation and considerably weaker at semantic recognition. Intersection over union measures both, since a misplaced boundary and a misnamed region both enter as false positives and false negatives. What the aggregate hides is not localisation but the balance between the two: the recognition deficit applies to most of the nineteen classes while the localisation advantage appears in three, so the mean is dominated by the former. The per-class table therefore admits no unqualified claim that the pretrained model is the better one.

Compute cost. Table 6 makes the cost comparison unusually clean, because the best accuracy-per-cost model is also the most accurate of the three-seed arms. The U-Net is dominated on every axis simultaneously: it holds 30% fewer parameters than DeepLabV3 yet takes 1.9 times as long per epoch, 1.5 times as long to serve one image, 2.7 times as long at batch 8, and 1.9 times the peak memory, while scoring 0.130 lower. Parameter count is simply not a proxy for cost here: the U-Net's convolutions run at full resolution and its decoder holds every encoder map alive for the skip connections, whereas MobileNetV3 reduces resolution early and keeps it reduced. The frozen arm is the one genuine trade: it trains in 0.84 hours against 1.52, a 45% saving, for 0.118 mIoU. Note that this saving is smaller than "freezing the backbone" suggests, because 73% of the parameters remain trainable, and that it is a saving in *training* only: inference cost is identical, 14.7 against 15.2 ms, since the same forward pass runs either way.

Scaling the backbone works, and is expensive. ResNet50 reaches 0.6665 validation and 0.7076 test mIoU, against MobileNetV3's 0.6136 and 0.6180. At one seed this supports no claim of superiority under the decision rule, and it is reported as indicative.The margin is nonetheless large relative to the noise: 0.0529 on validation, against a seed-to-seed standard deviation of 0.0051 measured for the same architecture under the same protocol. A tenfold ratio makes seed noise an implausible sole explanation. The cost is substantial: 5.3 times the seconds per epoch, 4.8 times the single-image latency, 9.4 times at batch 8, and 3.6 times the parameters, for roughly a 9% relative gain in test mIoU. On this hardware a single ResNet50 run costs more than the nine MobileNetV3 runs of the other three experiments combined.

Hypotheses. The model comparison is supported, with the per-class qualification above. The controlled experiment is supported on its main claim and refuted on its per-class prediction. The ablation is refuted outright, as Section 3.6 sets out. The backbone extension is consistent with its hypothesis but cannot establish it at one seed.

What does not follow. Three things. The U-Net's 0.4836 is a floor, not a converged score, so "pretraining beats from-scratch by 0.130" is an upper bound on that margin at this budget rather than an estimate of it. The comparison attributes that gap to pretraining, but the two arms differ in architecture as well, and no run here separates a DeepLabV3 trained from scratch from a pretrained one; that single experiment would isolate the factor properly and was not in the budget. And every number is a score at an untuned configuration under one shared protocol, so none of them is the best the corresponding method could reach.

3.5 Error analysis

The pretrained DeepLabV3 mislabels 15.37M of the 229.2M labelled test pixels, an error rate of 6.71% averaged over three seeds. Table 9 partitions every error pixel into six groups by the pair of classes involved, with precedence top to bottom so that each error falls in exactly one group.

Table 9: *Error taxonomy, as a share of each model's own errors.*

Group	DeepLabV3	ResNet50	Frozen	Dice	U-Net
Thin structures	24.7%	23.0%	19.6%	21.9%	18.7%
Upright surfaces	24.0%	26.5%	24.2%	24.0%	23.4%
Participant vs. background	17.5%	15.7%	16.5%	17.7%	15.8%
Ground surfaces	14.6%	14.8%	20.3%	17.4%	17.1%
Traffic participants	6.5%	6.8%	6.3%	6.4%	12.5%
Other	12.7%	13.3%	13.1%	12.7%	12.4%
Error rate	6.7%	4.9%	10.4%	10.2%	8.0%

The structure of the errors is a property of the task, not of the model. The five configurations differ by more than a factor of two in error rate, from 4.9% to 10.4%. Their error *composition*, however, is nearly identical. Thin structures account for between 19 and 25% of the errors for all five, upright surfaces between 23 and 27%, and ground surfaces between 15 and 20%. Improving the model lowers the total and leaves the proportions where they were. There is one exception, treated below.

Thin structures, 24.7%**.** Any error where a pole, traffic light or traffic sign is on either side. The largest single flows in the whole matrix sit here: pole predicted as building, 0.91M pixels or 5.9% of all errors, pole as vegetation, 0.56M or 3.6%, and traffic sign as building, 0.28M. Pole is the hardest class of the nineteen at 0.3230 IoU despite appearing in 99% of images, so this is not a data-scarcity failure: it is a resolution failure, and Figure 7 shows it directly. The cause is the one identified in Section 3.4: a stride-8 feature map upsampled bilinearly cannot represent a structure a few pixels wide, and a pole is mostly boundary. The evidence that this is architectural rather than inherent is that the from-scratch U-Net, with its full-resolution skip connections, scores 0.4361 on pole against DeepLabV3's 0.3230 while losing on almost everything else.

Upright surfaces, 24.0%**.** Confusions among building, wall, fence and vegetation, led by building predicted as vegetation, 1.16M pixels, and vegetation as building, 1.13M, which are nearly symmetric. These are overwhelmingly boundary pixels where a tree overhangs a facade, and the near-symmetry is the signature of a boundary error rather than a systematic bias. Wall and fence add a second cause:

both are low vertical barriers that the taxonomy separates from building by a convention rather than an appearance, and both are among the weakest classes at 0.4345 and 0.4689.

Ground surfaces, 14.6%**.** Road, sidewalk and terrain confused with one another, 1.12M pixels of sidewalk predicted as road and 0.74M the reverse. The boundary between road and sidewalk is a kerb a few pixels high, often occluded by parked cars, and at 512×1024 it is sometimes not visible at all. This group grows to 20.3% for the frozen arm, the largest compositional shift in the table.

Traffic participants, 6.5%**, and the one place a model differs in kind.** Confusions among the eight vehicle and person classes. This is the smallest group for every DeepLabV3 arm at between 6.3 and 6.8%, and it is 12.5% for the U-Net, nearly double. The cause is visible in the per-class table: the U-Net scores 0.0205 on rider, against DeepLabV3's 0.4316, and 0.0108 on motorcycle. Rider and person differ not in appearance but in what the person is sitting on, a distinction that requires recognising the vehicle beneath them, and the from-scratch model never learns it. This is the one error group where the deficit is a failure of recognition rather than of localisation, and it is where pretrained features help most.

Participant versus background, 17.5%**, and other,** 12.7%**.** The first is a person or vehicle confused with scenery, led by person predicted as building, 0.22M, and road predicted as car, 0.21M; these are object boundaries and shadow regions. The remainder is thinly spread and dominated by vegetation against terrain, 0.25M, and building against sky, 0.22M, both again boundary pairs.

What would improve it. Five of the six groups are dominated by boundary pixels, which points at resolution and not at the objective or the data. Three ideas follow, in descending order of expected return. Evaluating at the native 1024×2048 rather than the resized 512×1024 would halve the boundary length measured in pixels, and the resize is a choice made for compute, not for accuracy. An architecture that keeps a full-resolution path, which the U-Net's advantage on pole suggests concretely, would address the thin-structure group that is the single largest; a DeepLabV3+ decoder is the standard form of this. Only the traffic-participant group would benefit from more or better data rather than more resolution, and it is the smallest. What is *not* indicated is a reweighted loss: Section 3.6 measures that directly and it makes matters worse.

3.6 Ablation study

The ablation replaces cross-entropy with the soft Dice loss of Equation 3, holding the model, the data and every other protocol value fixed. The hypothesis, fixed in advance, was that Dice would raise IoU on train, bus and truck, the classes that are rare but large when present, because it weights classes rather than pixels; and that it would not help pole or traffic sign, whose errors are boundaries. The pre-registered criterion was that the hypothesis holds only if the per-class rule separates the two arms on at least two of train, bus and truck.

The hypothesis is refuted, and in the opposite direction. Table 10 applies the rule. The two arms are separated on train and on truck, so the criterion is met in form, but Dice is *worse* on both, and worse on every other class tested. Dice is worse overall, 0.4958 ± 0.0045 against 0.6136 ± 0.0051 validation mIoU, and worse on all nineteen classes individually.

Table 10: *The ablation, on validation IoU.*

Class	Cross-entropy	Dice	Change	P(absent from batch)
train	0.6007 ± 0.0383	0.0000 ± 0.0000	-0.6007	85.1%
bus	0.4154 ± 0.0923	0.1944 ± 0.1696	-0.2211	70.0%
truck	0.3797 ± 0.0398	0.1952 ± 0.0233	-0.1845	69.3%
motorcycle	0.4661 ± 0.0064	0.2775 ± 0.0313	-0.1886	60.2%
traffic sign	0.5518 ± 0.0036	0.4692 ± 0.0068	-0.0826	0.6%
pole	0.3277 ± 0.0085	0.3048 ± 0.0072	-0.0229	0.0%
road	0.9722 ± 0.0004	0.9502 ± 0.0016	-0.0220	0.0%

Why, and the mechanism is measurable. The last column of Table 10 is the key. Across all nineteen classes the correlation between the probability that a class is absent from a training batch and the damage Dice does to that class is $r = -0.809$: the more often a class is missing from the batch, the more Dice costs it. The four worst-damaged classes are exactly the four most often absent.

The cause lies in how the two objectives treat a class with no positive pixels in the batch. Under cross-entropy such a class contributes nothing of its own; it is only ever a competitor in the softmax denominator. Under Equation 3 it contributes a full term, $(0+\varepsilon)/(\sum_\Omega p_{cij}+0+\varepsilon)$, which is maximised by driving the predicted probability of that class to zero everywhere in the batch. Dice therefore *actively suppresses* every class that happens to be missing from the current eight crops. Measured over sampled batches, 10.43 of the 19 classes appear in a given 512×512 crop and 15.92 in a batch of eight, so 16.2% of the Dice terms in a typical step are suppression terms. For train, which appears in 2% of crops, roughly 85% of the 301 steps in an epoch push its logit down and about 45 steps push it up. The result is the only exact zero in this report: under Dice the model never predicts a single train pixel, on any seed, on either split.

This also explains the training behaviour. The Dice arm peaked at epoch 4 on all three seeds and early stopping ended it at epoch 12, with a training-to-validation gap of -0.002. It is not overfitting and it is not underfitting in the usual sense: the objective reaches a state where the suppression terms balance the overlap terms and stops producing useful gradient. The Dice arm is also the only one that is *more* expensive per epoch than its cross-entropy counterpart, 150.5 against 140.4 seconds, since it computes a softmax and per-class sums; it is cheapest overall only because it quits early.

What this does and does not establish. It establishes that an unweighted multi-class soft Dice with $\varepsilon = 1$, computed per batch at batch size 8, is the wrong objective for this class distribution, and it identifies the reason precisely enough to act on. It does not establish that class-balanced objectives are useless here. Three variants avoid the specific mechanism measured above: accumulating Dice over an epoch rather than a batch, masking absent classes out of the mean, and a class-weighted cross-entropy, which adds no suppression term at all. None of the three was run. The ablation revises the direction of Section 3.5's recommendations rather than supporting them: the remedy for rare classes is not to reweight the loss as implemented here.

3.7 Limitations and conclusion

Six limits bound these results. *The U-Net is undertrained:* its validation mIoU was still rising at epoch 40 on all three seeds, so 0.4836 is a floor and the margin over it an upper bound. *The model comparison confounds two factors:* its arms differ in architecture as well as in pretraining, so the gap is attributable to the pair, not to pretraining alone. *The backbone extension is one seed* and supports no claim of superiority. *Three seeds estimate a standard deviation from three observations,* so "indistinguishable" means not separated at this sample size, not equal. *The data is narrow:* daylight, fair weather, European cities and one camera, and the images were resized to a quarter of their pixels for compute, which is itself a ceiling on the thin classes that dominate the error analysis. *And reproducibility has a hardware limit:* MPS provides no deterministic kernel for the interpolation DeepLabV3 uses, so rerunning a DeepLabV3 seed will not reproduce its number exactly, and the reported spread includes that nondeterminism.

The research question was how much of the gap between a from-scratch model and a pretrained one comes from pretrained features rather than from architecture, loss or capacity. The evidence gives a sharp answer and an unexpected qualification. Fully fine-tuned pretrained features are worth 0.1301 validation mIoU over the from-scratch baseline, 0.1175 over the same weights with the backbone frozen, and 0.1178 over the same model trained with a class-balanced objective; and those three alternatives are mutually indistinguishable, within 0.0126 of one another. One factor accounts for essentially the whole spread. The qualification is that the advantage is not uniform: the from-scratch U-Net is better, with separated intervals, on pole, sky and vegetation, because a full-resolution decoder localises thin structures that a stride-8 backbone cannot, and five of the six error groups are dominated by boundary pixels. Pretraining supplies recognition rather than resolution.

DeepLabV3-MobileNetV3, fully fine-tuned, is carried forward at $\mathbf{0.6180 \pm 0.0049}$ test mIoU and $0.7441\pm$ 0.0048 test Dice, at 140 seconds per epoch and 15.2 ms per image.

Three directions follow, in the order the evidence supports them. The first is resolution: evaluating and training at the native 1024×2048, or adding a decoder with a full-resolution path, attacks the thin-structure and boundary groups that are most of the error. The second is the clean version of the model comparison, a DeepLabV3 trained from scratch, which would separate pretraining from architecture in

one run. The third is the class-balancing question the ablation reopened rather than closed: an epoch-accumulated Dice, or a Dice that masks absent classes out of its mean, avoids the suppression mechanism measured here and is the obvious next thing to try.

4 AI Usage Disclosure

Claude was used to check the candidate datasets against the handbook thresholds, to design the experiment plan and the fine-tuning procedure, to check the compute plan for the training runs, and to review the data loader for leakage. No prompt logs were retained, so each entry carries a prompt summary in their place, and no result in this report was accepted from a tool without the verification recorded below.

Tool	Claude Opus 5
Used by	Le Pham Tien Long
Stage	Dataset proposal (A2-M1)
Task	Checking the candidate datasets against the handbook thresholds
Prompt summary	Asked whether the six candidate datasets met the section 19 thresholds, and which to choose.
AI contribution	Checked each candidate against the thresholds; found one depth dataset short of the 5,000-pair floor at 1,449 labelled pairs, and a proposed Cityscapes link pointing at a resized subset rather than the official release.
Student verification	Re-read section 19 for every track, registered a Cityscapes account and confirmed the official packages, and reconciled the counts against each dataset's own documentation.
Affected files/sections	`PROPOSAL.md`, Section 1.2
Sources used for verification	The assignment handbook, sections 17, 18 and 19; the Cityscapes and NYU Depth dataset pages
Responsible member	Le Pham Tien Long

Tool	Claude Opus 5.5
Used by	Ho Minh Nhat
Stage	Experiment design (A2-M3)
Task	Designing the controlled experiment and the ablation
Prompt summary	Asked for an experiment plan giving each experiment a hypothesis, changed and fixed factors, metrics and a decision criterion.
AI contribution	Drafted the plan, confirmed every varied factor is already a configuration key the trainer reads, and derived the per-class hypotheses from the distribution analysis.
Student verification	Checked every protocol value against the base configuration, the data loader and the committed split, and the budget against the proposal's compute table.
Affected files/sections	`EXPERIMENTS.md`, Section 2.4
Sources used for verification	The assignment handbook, sections 20 and 21; `PROPOSAL.md`
Responsible member	Ho Minh Nhat

Tool	Claude Opus 5.5
Used by	Ho Minh Nhat
Stage	Fine-tuning procedure (A2-M2)
Task	Specifying the fine-tuning procedure
Prompt summary	Asked what the fine-tuning procedure still required against the existing code, then asked to complete it.
AI contribution	Recommended keeping the single-stage procedure of the proposal over staged unfreezing, moved the hardcoded flip probability into the configuration, and added tests holding the committed values.
Student verification	Ran the test suite on the branch and checked the documented procedure against the base configuration and the proposal before merging.
Affected files/sections	`configs/base.yaml`, `src/data.py`, Section 2.2
Sources used for verification	The assignment handbook, section 20; `PROPOSAL.md`, `configs/base.yaml`
Responsible member	Ho Minh Nhat

Tool	Claude Opus 5
Used by	Le Pham Tien Long
Stage	Compute planning (A2-M2)
Task	Checking the compute plan for the training runs
Prompt summary	Asked how many epochs each model could be trained for in the time remaining, and whether the planned budget was realistic.
AI contribution	Compared the per-epoch estimates against the rate the running job was achieving, found the U-Net 1.22 times slower than estimated, and tabulated the finishing time at 10, 20, 30 and 40 epochs.
Student verification	Checked the projected finishing time against the per-epoch timings recorded as the runs progressed, and confirmed the budget left margin for evaluation.
Affected files/sections	Section 2.3
Sources used for verification	The per-epoch timings recorded in each run's history; `PROPOSAL.md`
Responsible member	Le Pham Tien Long

Tool	Claude Opus 5
Used by	Le Pham Tien Long
Stage	Data pipeline (A2-M2)
Task	Reviewing the data loader for leakage
Prompt summary	Asked whether the data loading path could leak information between partitions, and whether the split holds the guarantees the proposal claims.
AI contribution	Traced the three routes by which leakage could enter and reported on each, and noted that masks must be resized by nearest neighbour, since interpolating label ids produces classes present in neither neighbour.
Student verification	Confirmed 2,406, 569 and 500 images against the real archives, that no city appears in two partitions, and that validation and test batches arrive uncropped. The labelId to trainId table was checked against `cityscapesscripts` rather than accepted, since a slip there would corrupt every later metric without failing.
Affected files/sections	`src/data.py`, `tests/test_data.py`, Section 1.4
Sources used for verification	The official Cityscapes toolkit `labels.py`; the committed split file; the dataset on disk
Responsible member	Le Pham Tien Long

Code Availability

Every deliverable for this assignment is linked from the assignment page, `https://longlephamtien.github.io/DL261-Assignment/assignments/assignment-2/`, and reachable directly below. Each reported number traces to a run directory holding its resolved configuration, its environment record and the commit it was produced at, as Table 11 sets out.

Source code	`https://github.com/longlephamtien/DL261-Assignment/tree/main/assignments/assignment-2`
Dataset proposal	`https://github.com/longlephamtien/DL261-Assignment/blob/main/assignments/assignment-2/PROPOSAL.md`
Experiment plan	`https://github.com/longlephamtien/DL261-Assignment/blob/main/assignments/assignment-2/EXPERIMENTS.md`
Checkpoints	`https://huggingface.co/longlephamtien/DL261-Assignment/tree/main/assignment-2/runs`
Slides	`https://github.com/longlephamtien/DL261-Assignment/blob/main/assignments/assignment-2/presentation/presentation.pdf`
Presentation video	`https://youtu.be/BBk91a9ARaM`

Table 11: *Traceability of every reported run.*

Configuration	Seed	Run ID	Best epoch	Commit
DeepLabV3-MobileNetV3	0	`deeplabv3-seed0-20261109-230158`	30	`567bab03`
DeepLabV3-MobileNetV3	1	`deeplabv3-seed1-20261110-003107`	31	`277d6794`
DeepLabV3-MobileNetV3	2	`deeplabv3-seed2-20261110-020225`	35	`277d6794`
frozen backbone	0	`deeplabv3-seed0-20261110-121645`	15	`277d6794`
frozen backbone	1	`deeplabv3-seed1-20261110-125324`	24	`277d6794`
frozen backbone	2	`deeplabv3-seed2-20261110-134416`	36	`277d6794`
Dice loss	0	`deeplabv3-seed0-20261110-144803`	4	`277d6794`
Dice loss	1	`deeplabv3-seed1-20261110-151818`	4	`277d6794`
Dice loss	2	`deeplabv3-seed2-20261110-154830`	4	`277d6794`
DeepLabV3-ResNet50	0	`deeplabv3-seed0-20261110-161840`	37	`277d6794`
U-Net	0	`unet-seed0-20261110-033604`	38	`277d6794`
U-Net	1	`unet-seed1-20261110-062947`	40	`277d6794`
U-Net	2	`unet-seed2-20261110-092324`	39	`277d6794`

References

[1] M. Cordts et al., "The cityscapes dataset for semantic urban scene understanding," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[3] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.

[4] A. Howard et al., "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[5] T.-Y. Lin et al., "Microsoft coco: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014.

[6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[8] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," in *Fourth International Conference on 3D Vision (3DV)*, 2016.

[9] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[10] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.